

Khung nghiên cứu đề xuất: Hybrid PCE–Dicke–VQA Portfolio Optimization

Kiến trúc phù hợp nhất với hướng nghiên cứu của bạn là một mô hình lai **hai tầng lượng tử – một tầng cổ điển**:

Dữ liệu thị trường thô



[1] Tiền xử lý cổ điển

Returns → Expected Returns → Covariance → Market Graph



[2A] PCE-based Universe Reduction, nếu số tài sản lớn

N tài sản → n tài sản ứng viên



[2B] QUBO → Ising Hamiltonian

Kèm ràng buộc cardinality/diversification



[3] Feasible-subspace Quantum Optimization

Dicke-VQE hoặc Dicke-QAOA/XY-QAOA



Classical Optimizer: CMA-ES / SPSA / PSO



[4] Measurement → Bitstrings → Asset Selection



Classical Continuous Weight Refinement



Rounding theo lot, kiểm tra ngân sách, transaction cost



Walk-forward Backtest và đánh giá

Điểm quan trọng là **Pauli Correlation Encoding** và **Dicke-state encoding** không nên được áp dụng đồng thời trên cùng một thanh ghi như hai cách mã hóa chồng lên nhau:

- PCE biểu diễn nhiều biến bằng dấu của các tương quan Pauli, phù hợp để giảm quy mô hoặc phân cụm một universe hàng trăm tài sản.
- Dicke state vẫn sử dụng gần như một qubit cho một biến lựa chọn, nhưng giới hạn trạng thái lượng tử trong không gian nghiệm thỏa mãn cardinality/diversification.

Vì vậy, đóng góp nghiên cứu hợp lý là sử dụng **PCE ở tầng tiền lựa chọn**, sau đó sử dụng **Dicke-VQE hoặc constraint-preserving QAOA ở tầng tối ưu chính xác hơn**. Soloviev và Krompiec đã sử dụng PCE để xử lý market graph hơn 250 biến, trong khi Scursulim và cộng sự sử dụng tensor product của các Dicke state để mã hóa trực tiếp ràng buộc đa lớp tài sản.

Bước 1. Data Preprocessing

1.1. Dữ liệu đầu vào

Với N tài sản và T thời điểm, dữ liệu ban đầu là ma trận giá:

$$P = \begin{bmatrix} P_{1,1} & \cdots & P_{1,N} & \vdots & \ddots & \vdots & P_{T,1} & \cdots & P_{T,N} \end{bmatrix}.$$

Nên sử dụng **adjusted close price** để phản ánh dividend, stock split và các corporate actions.

Quy trình xử lý nên gồm:

1. Đồng bộ ngày giao dịch giữa các tài sản.
2. Loại tài sản thiếu dữ liệu vượt quá ngưỡng, chẳng hạn 5–10%.
3. Không forward-fill qua những khoảng ngừng giao dịch dài.
4. Loại tài sản có thanh khoản thấp hoặc số ngày không thay đổi giá quá lớn.
5. Winsorize hoặc clipping các return cực đoan, nhưng phải thực hiện riêng trên training window để tránh look-ahead bias.

1.2. Ma trận tỷ suất sinh lời

Log-return:

$$r_{t,i} = \ln\left(\frac{P_{t,i}}{P_{t-1,i}}\right).$$

Ma trận return:

$$R = \begin{bmatrix} r_{1,1} & \cdots & r_{1,N} & \vdots & \ddots & \vdots & r_{T-1,1} & \cdots & r_{T-1,N} \end{bmatrix} \in R^{(T-1) \times N}.$$

Expected return đơn giản:

$$\mu_i = \frac{1}{T-1} \sum_{t=1}^{T-1} r_{t,i}.$$

Tuy nhiên, với dữ liệu tài chính, nên so sánh ít nhất ba cách:

$$\mu_i^{sample} = \frac{1}{T} \sum_t r_{t,i}$$

$$\mu_i^{EWMA} = \frac{\sum_{t=1}^T \delta^{T-t} r_{t,i}}{\sum_{t=1}^T \delta^{T-t}}, \quad 0 < \delta < 1,$$

và expected return từ mô hình dự báo cổ điển như ARIMA, XGBoost hoặc LSTM.

Để phân biệt đóng góp của phần lượng tử với sai số dự báo, mô hình chính nên bắt đầu bằng sample mean hoặc EWMA; predictive model có thể là thí nghiệm mở rộng.

1.3. Ma trận hiệp phương sai

Sample covariance:

$$\Sigma = \frac{1}{T-2} (R - \mathbf{1}\mu^\top)^\top (R - \mathbf{1}\mu^\top).$$

Trong thực nghiệm, nên sử dụng shrinkage:

$$\Sigma_{shrink} = (1 - \alpha)\Sigma_{sample} + \alpha F,$$

trong đó F có thể là diagonal covariance hoặc constant-correlation target.

Shrinkage giúp giảm tình trạng covariance matrix kém ổn định khi số tài sản lớn so với chiều dài của dữ liệu.

Phải kiểm tra:

$$\Sigma = \Sigma^\top, \quad \lambda_{min}(\Sigma) \geq 0.$$

Nếu xuất hiện eigenvalue âm nhỏ do sai số số học:

$$\Sigma_{PSD} = V \text{diag}(\max(\lambda_i, \varepsilon)) V^\top.$$

1.4. Chuẩn hóa hệ số trước khi tạo Hamiltonian

Return và covariance thường khác nhau đáng kể về thang đo. Có thể chuẩn hóa:

$$\mu^- = \frac{\mu}{\|\mu\|_\infty + \varepsilon}, \quad \Sigma^- = \frac{\Sigma}{\|\Sigma\|_\infty + \varepsilon}.$$

Khi đó tham số risk aversion λ có ý nghĩa rõ hơn:

$$f(x) = \lambda x^\top \Sigma^- x - (1 - \lambda) \mu^{-\top} x.$$

Sau khi xây dựng Ising Hamiltonian, tiếp tục chia toàn bộ hệ số cho:

$$s_H = \max\left(\max_i |h_i|, \max_{i,j} |J_{ij}|\right).$$

Việc này tránh cost angles quá lớn và cải thiện điều kiện số cho vòng lặp variational. Giá trị s_H phải được lưu lại để chuyển energy chuẩn hóa về objective ban đầu.

1.5. Market graph cho tầng PCE

Từ covariance, tính correlation:

$$\rho_{ij} = \frac{\Sigma_{ij}}{\sqrt{\Sigma_{ii} \Sigma_{jj}}}.$$

Xây dựng graph:

$$G = (V, E, W),$$

trong đó:

- mỗi node v_i là một tài sản;
- mỗi edge (i, j) biểu diễn quan hệ giữa tài sản i, j ;
- trọng số có thể là $w_{ij} = |\rho_{ij}|, \rho_{ij}^2$ hoặc chỉ giữ edge khi $|\rho_{ij}| > \rho_0$.

PCE có thể thực hiện recursive bipartition trên graph này để tách các nhóm tài sản tương quan cao, sau đó chọn representative asset từ từng nhóm. Đây cũng là cách tiếp cận trong nghiên cứu PCE: quyết định lượng tử chủ yếu xử lý cấu trúc tương quan, còn bước cổ điển phía sau xây dựng portfolio rút gọn dựa trên return và risk.

Pseudocode bước 1

```
def preprocess_prices(prices, annual_factor=252):
    # 1. Align trading dates and remove assets with excessive missingness
    prices = align_and_clean(prices)

    # 2. Compute Log returns
    returns = np.log(prices / prices.shift(1)).dropna()

    # 3. Robust treatment of extreme observations
    returns = winsorize_training_data(returns)

    # 4. Estimate return and covariance
    mu_daily = ewma_mean(returns, decay=0.97)
    sigma_daily = shrinkage_covariance(returns)

    # 5. Enforce positive semidefinite covariance
    sigma_daily = nearest_psd(sigma_daily, epsilon=1e-8)

    # 6. Annualize consistently
    mu = annual_factor * mu_daily
    sigma = annual_factor * sigma_daily

    # 7. Normalize quantum coefficients
    mu_scaled = mu / (np.max(np.abs(mu)) + 1e-12)
    sigma_scaled = sigma / (np.max(np.abs(sigma)) + 1e-12)

    # 8. Construct correlation-based market graph
    corr = covariance_to_correlation(sigma)
    graph = build_market_graph(corr)
```

```

return {
    "returns": returns,
    "mu": mu,
    "sigma": sigma,
    "mu_scaled": mu_scaled,
    "sigma_scaled": sigma_scaled,
    "correlation": corr,
    "market_graph": graph,
}

```

Bước 2. Quantum Encoding / Mapping

2.1. Biến quyết định

Đề xuất chính là để tăng lượng tử giải bài toán **asset selection**:

$$x_i = \{1, \text{ chọn tài sản } i, 0, \text{ không chọn tài sản } i\}.$$

Tỷ trọng liên tục được tối ưu ở bước hậu xử lý. Cách tách này tiết kiệm qubit đáng kể so với mã hóa trực tiếp từng trọng số bằng nhiều bit.

2.2. QUBO tổng quát

Một objective thực tế có thể được viết:

$$F(x) = \lambda x^\top \Sigma x - (1 - \lambda) \mu^\top x + \tau \sum_i c_i |x_i - x_i^{prev}| + A_0 \left(\sum_i x_i - K \right)^2 + \sum_{g=1}^G A_g \left(\sum_i a_{g,i} x_i - k_g \right)^2.$$

Trong đó:

- K : tổng số tài sản cần chọn;
- $a_{g,i} = 1$ nếu tài sản i thuộc nhóm g ;
- k_g : số tài sản cần chọn từ nhóm g ;
- τ : hệ số transaction cost hoặc turnover;
- x^{prev} : danh mục ở kỳ tái cân bằng trước.

Vì x_i^{prev} là hằng số:

$$|x_i - x_i^{prev}| = x_i + x_i^{prev} - 2x_i x_i^{prev},$$

nên transaction cost vẫn trở thành một hạng tuyến tính theo x_i .

Hard constraint hay penalty?

Nếu sử dụng Dicke state hoặc XY mixer:

$$\sum_i x_i = K$$

và

$$\sum_i a_{g,i} x_i = k_g$$

đã được đảm bảo bởi cấu trúc mạch. Khi đó nên **loại các penalty tương ứng khỏi Hamiltonian**:

$$F_{hard}(x) = \lambda x^T \Sigma^- x - (1 - \lambda) \mu^{-T} x + \tau C_{turnover}(x).$$

Điều này tránh phải hiệu chỉnh penalty coefficient và tránh làm méo energy landscape.

Nếu dùng QAOA với X-mixer thông thường hoặc quantum annealing, các penalty vẫn cần thiết. Nghiên cứu thực tế của Sakuler và cộng sự cho thấy việc hiệu chỉnh hệ số QUBO là một thành phần quan trọng và đề xuất một cơ chế hybrid để tự động tuning các tham số này.

2.3. Chuyển QUBO thành Ising Hamiltonian

Giả sử QUBO đã được khai triển thành:

$$F(x) = c + \sum_i a_i x_i + \sum_{i < j} b_{ij} x_i x_j.$$

Chọn quy ước:

$$x_i = \frac{I - Z_i}{2}.$$

Khi đó:

$$x_i x_j = \frac{I - Z_i - Z_j + Z_i Z_j}{4}.$$

Hamiltonian tương ứng là:

$$H_C = c_0 I + \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j,$$

với:

$$J_{ij} = \frac{b_{ij}}{4},$$

$$h_i = -\frac{a_i}{2} - \frac{1}{4} \sum_{j \neq i} b_{ij},$$

$$c_0 = c + \frac{1}{2} \sum_i a_i + \frac{1}{4} \sum_{i < j} b_{ij}.$$

Hằng số c_0 không ảnh hưởng đến bitstring tối ưu nên có thể bỏ khỏi mạch, nhưng nên giữ lại khi báo cáo objective value.

Hamiltonian phải thỏa:

$$H_c|x\rangle = F(x)|x\rangle.$$

Pseudocode QUBO $\rightarrow$ Ising

```
def qubo_to_ising(linear, quadratic, constant=0.0):
    h = defaultdict(float)
    J = defaultdict(float)
    offset = constant

    for i, a_i in linear.items():
        offset += a_i / 2.0
        h[i] -= a_i / 2.0

    for (i, j), b_ij in quadratic.items():
        offset += b_ij / 4.0
        h[i] -= b_ij / 4.0
        h[j] -= b_ij / 4.0
        J[(i, j)] += b_ij / 4.0

    return h, J, offset
```

2.4. Nhánh Pauli Correlation Encoding

PCE sử dụng:

$$s_i = \text{sgn}(\langle \psi(\theta) | \Pi_i | \psi(\theta) \rangle) \in \{-1, +1\},$$

sau đó chuyển sang biến nhị phân:

$$x_i = \frac{1-s_i}{2}.$$

Với n_q qubit và Pauli correlation order k , số biến có thể được mã hóa tăng theo bậc đa thức $O\left(n_q^k\right)$, thay vì một biến cho một qubit. Ví dụ, với ba qubit và $k = 2$, các observables có thể bao gồm $XXI, XIX, IXX, YYI, YIY, IYY, ZZI, ZIZ, IZZ$.

Trong framework đề xuất, PCE nên làm nhiệm vụ:

```

Large asset universe
↓
Correlation graph
↓
PCE recursive graph bipartition
↓
Clusters of redundant/correlated assets
↓
Select representatives or top candidates per cluster
↓
Reduced universe for Dicke-VQE/QAOA

```

PCE không tạo bitstring bằng phép đo từng qubit. Vì vậy, direct QUBO-to-Ising mapping ở phần trên không được áp dụng nguyên trạng cho PCE; objective PCE được tính cổ điển từ các Pauli correlations đã đo.

Pseudocode PCE preselection

```

def pce_recursive_preselection(graph, target_size, quantum_solver):
    clusters = [graph.nodes]

    while total_representatives(clusters) > target_size:
        new_clusters = []

        for cluster in clusters:
            if should_split(cluster, target_size):
                pauli_map = construct_pauli_correlations(
                    num_variables=len(cluster),
                    correlation_order=choose_order(len(cluster))
                )

                theta_star = quantum_solver.optimize_cut(
                    graph.subgraph(cluster),
                    pauli_map
                )

                correlations = quantum_solver.measure_paulis(
                    theta_star,
                    pauli_map
                )

                binary_partition = {
                    node: int(correlations[node] < 0)
                    for node in cluster
                }

                left, right = split_cluster(cluster, binary_partition)
                new_clusters.extend([left, right])
            else:
                new_clusters.append(cluster)

```



```

clusters = new_clusters

candidates = [
    choose_representative_asset(cluster)
    for cluster in clusters
]

return candidates

```

Với $|\langle \Pi_i \rangle|$ gần 0, quyết định dấu không ổn định. Nên đánh dấu các biến có:

$$|\langle \Pi_i \rangle| < \epsilon_{PCE}$$

là ambiguous và xử lý bằng multiple restarts hoặc classical tie-breaking.

2.5. Dicke-state encoding cho ràng buộc

Dicke state với n qubit và Hamming weight k :

$$\left| D_n^k \right\rangle = \frac{1}{\sqrt{\binom{n}{k}}} \sum_{x \in \{0,1\}^n, |x|=k} \left| x \right\rangle.$$

Mọi bitstring trong trạng thái này đều thỏa:

$$\sum_i x_i = k.$$

Nếu có G nhóm tài sản:

$$\left| \psi_0 \right\rangle = \bigotimes_{g=1}^G \left| D_{n_g}^{k_g} \right\rangle.$$

Không gian tìm kiếm giảm từ:

$$2^N$$

xuống:

$$|F| = \prod_{g=1}^G \binom{n_g}{k_g}.$$

Nghiên cứu của Scursulim và cộng sự cho thấy cấu trúc này loại các nghiệm bất khả thi khỏi không gian tìm kiếm, không cần penalty cho diversification constraints và đạt kết quả tốt nhất trong thí nghiệm của họ khi kết hợp với CMA-ES.

Bước 3. Quantum Optimization Algorithm

3.1. Phương án đề xuất chính: Dicke-VQE

Trạng thái variational:

$$|\psi(\theta)\rangle = U_D(\theta)|\psi_0\rangle.$$

Một ansatz bảo toàn Hamming weight có thể sử dụng:

$$U_D(\theta) = \prod_{\ell=1}^L \left[\prod_i R_Z(\phi_{\ell i}) \prod_{(i,j) \in E_M} \exp\left(-\frac{i\theta_{\ell ij}}{2}(X_i X_j + Y_i Y_j)\right) \right].$$

Các XY-rotation chỉ hoán đổi excitation giữa các qubit nên bảo toàn số bit 1.

Loss function:

$$L(\theta) = \langle \psi(\theta) | H_c | \psi(\theta) \rangle.$$

Do H_c chỉ chứa Z_i và $Z_i Z_j$, tất cả các hạng đều commute. Vì vậy, một tập measurements trong computational basis có thể dùng để ước lượng toàn bộ energy:

$$\hat{L}(\theta) = \sum_z \hat{p}_\theta(z) F(z).$$

Điều này đơn giản hơn đáng kể so với một Hamiltonian có các hạng X, Y không commute.

3.2. Classical optimizer

Vòng lặp hybrid:

```
θ
↓
Prepare feasible quantum state
↓
Execute circuit with S shots
↓
Estimate E(θ)=⟨H⟩
↓
Classical optimizer proposes θnew
↓
Repeat until convergence
```

Lựa chọn optimizer nên được xem là một biến thực nghiệm:

- **CMA-ES:** phù hợp với landscape phức tạp, derivative-free; đạt kết quả tốt trong nghiên cứu Dicke-VQE của Scursulim.
- **SPSA/QNSPSA:** phù hợp khi có shot noise và hardware noise.

- **COBYLA:** baseline đơn giản cho số tham số vừa phải.
- **PSO:** đáng thử cho QAOA/QAOAz; một nghiên cứu QAOAz-portfolio năm 2025 báo cáo PSO cải thiện quá trình tuning circuit parameters trong các thí nghiệm mean-variance và risk-parity.
- **L-BFGS-B:** chủ yếu dùng cho statevector hoặc analytic-gradient simulation.

Một chiến lược tốt là:

Multi-start CMA-ES

↓

Chọn 3-5 nghiệm tốt nhất

↓

SPSA fine-tuning trên shot-based simulator/hardware

3.3. Pseudocode Dicke-VQE

```
def dicke_vqe(cost_hamiltonian, groups, selected_per_group,
              optimizer, max_iter, shots):
```

```
    # Tensor product of group-wise Dicke states
```

```
    initial_state = prepare_multiclass_dicke_state(
        groups,
        selected_per_group
    )
```

```
    ansatz = build_number_preserving_ansatz(
        initial_state=initial_state,
        mixer="XY",
        topology="ring",
        layers=3
    )
```

```
    theta = initialize_parameters(ansatz)
```

```
    best = {
        "theta": theta,
        "energy": float("inf"),
    }
```

```
    for iteration in range(max_iter):
        counts = execute_and_measure(
            ansatz,
            theta,
            shots=shots,
            basis="Z"
        )
```

```
        energy = expectation_from_counts(
            counts,
            cost_hamiltonian
        )
```

```

    )

    if energy < best["energy"]:
        best = {
            "theta": theta.copy(),
            "energy": energy,
        }

    theta = optimizer.update(
        parameters=theta,
        objective_value=energy
    )

    if converged(energy):
        break

    final_counts = execute_and_measure(
        ansatz,
        best["theta"],
        shots=10 * shots,
        basis="Z"
    )

    return best, final_counts

```

3.4. Phương án thay thế: Constraint-preserving QAOA

Trạng thái QAOA:

$$\left| \Psi(\gamma, \beta) \right\rangle = \prod_{\ell=1}^p e^{-i\beta_{\ell} H_M} e^{-i\gamma_{\ell} H_C} \left| \Psi_0 \right\rangle.$$

Cost Hamiltonian:

$$H_C = \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j.$$

Mixer bảo toàn Hamming weight:

$$H_M = \frac{1}{2} \sum_{g=1}^G \sum_{(i,j) \in E_g} (X_i X_j + Y_i Y_j).$$

Chỉ kết nối các qubit trong cùng nhóm tài sản, do đó từng k_g được bảo toàn.

Có hai lựa chọn topology:

$$E_g^{ring} = \{(1, 2), (2, 3), \dots, (n_g, 1)\},$$

hoặc complete graph:

$$E_g^{complete} = \{(i, j): i < j\}.$$

Ring mixer có circuit depth thấp hơn. Complete mixer có connectivity tốt hơn nhưng đòi hỏi nhiều two-qubit gates.

Quantum-walk mixer cũng có thể được dùng như một thí nghiệm mở rộng. Nghiên cứu QWOA trong bộ tài liệu của bạn cho thấy việc giới hạn evolution vào các nghiệm hợp lệ, không suy biến, có thể giảm search space và tăng tốc độ hội tụ so với QAOA thông thường, đổi lại indexing circuit phức tạp hơn.

Pseudocode QAOA

```
def feasible_qaoa(H_cost, H_mixer, initial_dicke_state,
                  depth_p, optimizer, shots):

    gamma = initialize_gamma(depth_p)
    beta = initialize_beta(depth_p)

    while not stopping_condition():
        circuit = initial_dicke_state.copy()

        for layer in range(depth_p):
            circuit.apply_cost_unitary(
                H_cost,
                angle=gamma[layer]
            )
            circuit.apply_mixer_unitary(
                H_mixer,
                angle=beta[layer]
            )

        counts = circuit.sample(shots=shots)
        loss = exact_qubo_expectation(counts)

        gamma, beta = optimizer.update(
            gamma,
            beta,
            loss
        )

    return circuit.sample(shots=10 * shots)
```

3.5. Layerwise training và warm-start

Không nên tối ưu trực tiếp một QAOA sâu từ tham số ngẫu nhiên.

Quy trình layerwise:

Tối ưu $p = 1$



Dùng (γ_1, β_1) làm warm-start cho $p = 2$



Khởi tạo layer mới bằng giá trị nhỏ



Tối ưu $p = 2$



Tiếp tục đến $p_{\max}$

Tương tự, khi tái cân bằng danh mục ở thời điểm $t + 1$, có thể dùng tham số tối ưu ở thời điểm t làm warm-start vì μ và Σ thường không thay đổi hoàn toàn giữa hai kỳ liên tiếp.

3.6. Quantum annealing làm benchmark

Nhánh annealing nhận trực tiếp QUBO:

$$\min_{x \in \{0,1\}^N} F(x).$$

Pipeline:

QUBO



Binary Quadratic Model



Minor embedding / hybrid decomposition



Forward or reverse annealing



Sample bitstrings



Classical local search / penalty tuning

Sakuler và cộng sự giải một QUBO từ bài toán danh mục thực tế bằng hai D-Wave hybrid solvers và một solver cổ điển, đạt nghiệm phù hợp với global optimum cổ điển; nghiên cứu cũng nhấn mạnh vai trò của penalty tuning.

Một nghiên cứu end-to-end năm 2025 cũng sử dụng mô hình quantum-classical trong đó quantum annealing thực hiện asset selection còn các bước allocation và rebalancing được tích hợp trong pipeline cổ điển-lượng tử.

Bước 4. Post-processing & Decoding

4.1. Giải mã bitstring

Với measurement result:

$$C = \left\{ \left(z^{(1)}, n_1 \right), \dots, \left(z^{(M)}, n_M \right) \right\},$$

xác suất thực nghiệm:

$$\hat{p}(z) = \frac{n_z}{S}.$$

Không nên chỉ chọn bitstring có xác suất cao nhất. Nên:

1. Lấy top- L bitstrings theo xác suất.
2. Kiểm tra constraint.
3. Tính lại objective cổ điển chính xác $F(z)$.
4. Chọn bitstring có objective tốt nhất trong tập đã đo.

$$x^* = \arg \min_{z \in Z_{measured} \cap F} F(z).$$

Điều này quan trọng vì bitstring tốt nhất về objective không nhất thiết là bitstring có frequency cao nhất.

4.2. Feasibility repair

Với Dicke hoặc XY mixer lý tưởng, feasibility rate phải gần 100%. Trên hardware, noise có thể làm sai Hamming weight.

Nếu:

$$\sum_i x_i \neq K,$$

có thể repair bằng cách tìm nghiệm khả thi gần nhất:

$$\tilde{x} = \arg \min_{y \in F} d_H(x, y),$$

trong đó d_H là Hamming distance.

Với ràng buộc theo nhóm, repair riêng từng group:

```
for group in asset_groups:
    if number_selected(group) > k[group]:
        remove assets with lowest marginal contribution

    if number_selected(group) < k[group]:
        add assets with best marginal contribution
```

Marginal contribution có thể được tính bằng:

$$\Delta_i F = F(x \oplus e_i) - F(x).$$

4.3. Chuyển asset selection thành tỷ trọng

Trường hợp equal-weight

Nếu mục tiêu chỉ là chọn K tài sản:

$$w_i = \frac{x_i}{K}.$$

Trường hợp thực tế: classical weight refinement

Đặt:

$$S = \{i: x_i^* = 1\}.$$

Sau đó giải bài toán liên tục trên tập S :

$$\min_{w_S} \quad \lambda_w w_S^\top \Sigma_S w_S - (1 - \lambda_w) \mu_S^\top w_S + \kappa \|w_S - w_S^{prev}\|_2^2 \text{ s.t. } \quad 1^\top w_S = 1, \quad l_i \leq w_i \leq u_i, \quad w_i = 0, \quad i \notin S.$$

Đây là convex quadratic program nếu:

- $\Sigma_S \succcurlyeq 0$;
- chỉ có long positions;
- các constraints là tuyến tính.

Kiến trúc này phân chia vai trò rõ ràng:

Quantum component:
tìm tổ hợp tài sản tốt

Classical component:
tìm tỷ trọng vốn chính xác

Nó cũng phù hợp với giới hạn qubit hiện tại hơn so với fixed-point encoding toàn bộ weights.

4.4. Chuyển trọng số thành số lượng cổ phiếu

Với tổng vốn B , giá hiện tại P_i :

$$q_i^{(0)} = \lfloor \frac{Bw_i}{P_i} \rfloor.$$

Giá trị đầu tư:

$$V_{used} = \sum_i q_i^{(0)} P_i.$$

Phần vốn còn lại:

$$B_{remain} = B - V_{used}.$$

Phân phối phần còn lại theo fractional remainder lớn nhất hoặc marginal improvement tốt nhất, đồng thời tính transaction fee:

$$C_{trade} = \sum_i c_i |q_i - q_i^{prev}|.$$

4.5. Pseudocode hệ xử lý

```
def decode_and_allocate(counts, qubo_objective,
                        constraints, mu, sigma,
                        previous_weights=None):

    # 1. Aggregate and rank measured states
    measured_states = aggregate_counts(counts)
    top_states = select_top_states(measured_states, top_k=100)

    # 2. Repair noisy infeasible states if necessary
    candidates = []
    for bitstring, probability in top_states:
        x = bitstring_to_binary_vector(bitstring)

        if not constraints.is_feasible(x):
            x = repair_to_nearest_feasible(x, constraints)

        exact_cost = qubo_objective(x)

        candidates.append({
            "x": x,
            "probability": probability,
            "cost": exact_cost,
        })

    # 3. Re-rank by the original objective, not only frequency
    best_selection = min(candidates, key=lambda item: item["cost"])
    selected_assets = np.where(best_selection["x"] == 1)[0]

    # 4. Continuous weight optimization
    weights = solve_continuous_mean_variance_qp(
        selected_assets=selected_assets,
        expected_returns=mu,
        covariance=sigma,
        previous_weights=previous_weights,
        budget_constraint=1.0,
        long_only=True
    )
```

```
# 5. Optional conversion to tradable integer lots
```

```
lots = convert_weights_to_integer_lots(weights)
```

```
return {  
    "selection": best_selection["x"],  
    "weights": weights,  
    "lots": lots,  
    "quantum_probability": best_selection["probability"],  
    "qubo_cost": best_selection["cost"],  
}
```

Pseudocode end-to-end cho toàn bộ framework

```
def hybrid_quantum_portfolio_pipeline(  
    prices,  
    asset_classes,  
    selected_per_class,  
    rebalance_dates,  
    large_universe_threshold,  
    quantum_backend,  
):  
  
    previous_weights = None  
    portfolio_history = []  
  
    for rebalance_date in rebalance_dates:  
  
        # =====  
        # STEP 1: DATA PREPROCESSING  
        # =====  
        train_prices = get_rolling_window(  
            prices,  
            end_date=rebalance_date,  
            lookback_days=504  
        )  
  
        data = preprocess_prices(train_prices)  
  
        mu = data["mu"]  
        sigma = data["sigma"]  
        mu_scaled = data["mu_scaled"]  
        sigma_scaled = data["sigma_scaled"]  
  
        # =====  
        # STEP 2A: PCE UNIVERSE REDUCTION  
        # =====  
        universe = list(train_prices.columns)
```

```

if len(universe) > large_universe_threshold:
    candidates = pce_recursive_preselection(
        graph=data["market_graph"],
        target_size=large_universe_threshold,
        quantum_solver=quantum_backend
    )
else:
    candidates = universe

mu_q = mu_scaled[candidates]
sigma_q = sigma_scaled[np.ix_(candidates, candidates)]

class_constraints = build_class_constraints(
    candidates,
    asset_classes,
    selected_per_class
)

# =====
# STEP 2B: QUBO AND HAMILTONIAN
# =====
qubo = build_selection_qubo(
    expected_returns=mu_q,
    covariance=sigma_q,
    risk_aversion=0.5,
    previous_weights=previous_weights,
    hard_cardinality=True
)

H_cost = qubo_to_ising_hamiltonian(qubo)

# =====
# STEP 3: QUANTUM OPTIMIZATION
# =====
initial_state = tensor_product_dicke_states(
    class_constraints
)

ansatz = number_preserving_xy_ansatz(
    initial_state,
    layers=3,
    topology="ring"
)

theta_star = optimize_vqe(
    ansatz=ansatz,
    hamiltonian=H_cost,
    optimizer="CMA-ES_then_SPSA",
    shots=4096,

```

```

        multi_start=20
    )

    counts = sample_final_circuit(
        ansatz,
        theta_star,
        shots=20000
    )

    # =====
    # STEP 4: DECODING AND WEIGHT REFINEMENT
    # =====
    solution = decode_and_allocate(
        counts=counts,
        qubo_objective=qubo.evaluate,
        constraints=class_constraints,
        mu=mu[candidates],
        sigma=sigma[np.ix_(candidates, candidates)],
        previous_weights=previous_weights
    )

    previous_weights = solution["weights"]

    portfolio_history.append({
        "date": rebalance_date,
        **solution
    })

    return walk_forward_evaluation(portfolio_history, prices)

```

Thiết kế thí nghiệm nghiên cứu

Để mô hình có giá trị khoa học, nên tách thành các nhóm ablation sau:

- M0: Classical MIQP / SCIP
- M1: Classical heuristic hoặc simulated annealing
- M2: Standard QAOA + X-mixer + penalties
- M3: XY-QAOA + feasible Dicke initial state
- M4: Dicke-VQE
- M5: PCE preselection + Dicke-VQE
- M6: Quantum annealing / hybrid annealing

Các metrics nên chia thành ba nhóm.

Chất lượng tối ưu

$$Approximation\ ratio = \frac{E_{max} - E(\theta^*)}{E_{max} - E_0}.$$

Ngoài ra:

- best feasible objective;
- feasibility rate;
- probability của nghiệm tối ưu;
- probability của top-1%, top-5%;
- number of optimizer evaluations;
- convergence variance qua nhiều random seeds.

Tài nguyên lượng tử

- số qubit;
- circuit depth;
- số two-qubit gates;
- số shots;
- số measurement groups;
- wall-clock execution time;
- time-to-solution;
- hiệu năng khi có noise.

Hiệu quả tài chính out-of-sample

$$R_p = w^\top \mu,$$

$$\sigma_p = \sqrt{w^\top \Sigma w},$$

$$Sharpe = \frac{R_p - r_f}{\sigma_p}.$$

Kèm theo:

- cumulative return;
- annualized volatility;
- Sortino ratio;
- maximum drawdown;
- turnover;
- transaction cost;
- stability của selected assets;
- tracking error nếu có benchmark.

Nên sử dụng rolling hoặc walk-forward evaluation:

Training window

→ Optimize

→ Hold portfolio trong test period

→ Rebalance

- Trượt cửa sổ
- Lặp lại

Không nên tuyên bố quantum advantage chỉ từ objective tốt hơn ở một vài bộ dữ liệu. Một benchmark quy mô lớn gần đây cho thấy mixed-integer programming và heuristic chuyên biệt vẫn có thể vượt các phương pháp quantum hiện tại trên nhiều instance portfolio; vì vậy đóng góp nên được diễn đạt là **qubit efficiency, constraint feasibility, search-space reduction và hybrid workflow improvement**, trừ khi có bằng chứng scaling rõ ràng.

Cấu hình mô hình đề xuất để triển khai đầu tiên

Universe ban đầu: 100–300 tài sản
PCE preselection: giảm còn 15–30 tài sản
Asset classes: 3–5 nhóm
Dicke constraint: chọn 2–5 tài sản mỗi nhóm
Quantum core: VQE
Ansatz: parametrized Dicke + number-preserving XY rotations
Circuit layers: $L = 2, 3, 4$
Optimizer: CMA-ES → SPSSA
Shots khi training: 2,048–4,096
Shots cuối: 10,000–20,000
Weight refinement: classical convex QP
Backtest: rolling 24 tháng train / 1 tháng test
Baselines: SCIP, classical QP, SA, standard QAOA, QA

Cấu hình này kết hợp được ba ưu điểm chính: **khả năng mở rộng của PCE, khả năng thỏa ràng buộc nội tại của Dicke state, và tính thực dụng của bước tối ưu tỷ trọng cổ điển sau asset selection.**